

Sistemas Operacionais

Atividade Prática 01 — Processos e Mapeamento de Memória

Aluno(a): _____

Matrícula: _____ Turma: _____

Data: _____

Instruções. Faça as atividades dentro do laboratório Linux (container `so-lab`: `./so.sh`). Para cada questão, execute os comandos indicados, observe a saída e responda de forma objetiva. Onde aparecer `<PID>`, use o número que apareceu na sua tela.

PARTE A · PROCESSOS

1 Processo, PID e processo-pai (PPID)

2,0 pts

Coloque um processo para dormir em segundo plano e observe sua identificação:

```
so:/so$ sleep 200 &
so:/so$ ps -o pid,ppid,stat,cmd -C sleep
```

Responda:

- a) Qual é o **PID** do `sleep` e qual é o **PPID** (o processo que o criou)?
- b) Rode também `echo $$`. O que esse número tem a ver com o PPID acima?

2 A chamada `fork()` cria um novo processo

2,0 pts

Compile e execute o programa abaixo:

```
fork.c

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void){
    pid_t f = fork();
    if (f == 0) printf("Sou o FILHO (PID %d)\n", getpid());
    else { wait(NULL); printf("Sou o PAI (PID %d), filho = %d\n", getpid(), f); }
    return 0;
}
```

```
so:/so$ gcc -Wall -o fork fork.c && ./fork
```

Responda:

- a) A mensagem apareceu **quantas vezes**? Por quê, se o programa tem um só `printf` em cada ramo?
- b) Qual valor o `fork()` devolveu ao **pai** e qual devolveu ao **filho**? (dica: o teste `f == 0`)

3 Estados de um processo

2,0 pts

Observe como o estado de um processo muda quando ele recebe sinais:

```
so:/so$ sleep 300 &
so:/so$ ps -o stat= -p <PID>          # estado inicial
so:/so$ kill -STOP <PID> ; ps -o stat= -p <PID>
so:/so$ kill -CONT <PID> ; ps -o stat= -p <PID>
```

Responda:

- a) Qual o estado inicial do `sleep` e o que essa letra significa?
- b) Para que estado ele vai após o `-STOP`? E depois do `-CONT`?

PARTE B · MAPEAMENTO DE MEMÓRIA

4 As regiões de memória de um processo

2,0 pts

Use o programa `mem.c` (com `malloc` e `sleep(30)`). Com ele rodando, inspecione o mapa:

```
so:/so$ gcc -Wall -g -o mem mem.c
so:/so$ ./mem &
so:/so$ cat /proc/<PID>/maps | grep -E 'mem|heap|stack'
```

Responda:

- a) Escreva a faixa de endereços da linha marcada como `[heap]` e da linha `[stack]`.
- b) A linha do binário com permissão `r-xp` guarda o quê: dados, pilha ou código? Justifique pela permissão.

5 Onde mora cada variável

2,0 pts

No trecho abaixo, cada variável fica em uma região diferente da memória do processo:

```
mem.c (trecho)

int x = 10;                // variável global
int main(){
    int y = 20;            // variável local
    int *p = malloc(sizeof(int)); // ponteiro + reserva
    *p = 50;
    ...
}
```

Associe cada item à sua região (escreva ao lado): DADOS, STACK (pilha) ou HEAP.

- () a variável global `x` • () a variável local `y`
- () o ponteiro `p` em si • () o valor `50` apontado por `*p`

Explique em uma frase por que `p` e `*p` ficam em regiões diferentes: